

Porting Tool Manual

Contents

1. Purpose of Porting Tool.....	3
2. File Structure of Tool.....	3
3. Installation and Uninstallation.....	3
4. Overall Operation of Tool.....	4
5. Configuration of Tool's Main Frame Window.....	5
6. Operation Procedure.....	6
6.1 Opening file.....	6
6.2 How to make settings.....	6
6.3 Executing Tool.....	9
7. Description of Tool's Functions (LSI C)	11
8. Conversion Items of Porting Tool (LSI C)	12
8.1 Conversion of FAR declaration.....	12
8.2 Conversion of FAR FUNCTION.....	13
8.3 Conversion of NULL.....	13
8.4 Conversion of "stdrom.h".....	14
8.5 Conversion of header file.....	14
8.6 Conversion of MACRO.....	15
8.7 Conversion of BAD CODE.....	15
8.8 Conversion of compiler order "#endif".....	16
8.9 Conversion of unjust semicolon.....	16
9. Notes for Porting Tool (LSI C)	17
9.1 Output of warning due to incompatibility of type declarations.....	17
9.2 Treatment of floating-point constant.....	19
9.3 Use of assembly language.....	19
9.4 Initialization of self-reference struct.....	20
9.5 Position to insert cast conversion.....	21
9.6 Initialization of strings.....	22
9.7 FP_OFF(), FP_SEG() and MK_FP() - segmentation/composition of far pointer.....	23
Appendix.....	24

1. Purpose of Porting Tool

This is a document about the Porting Tool (hereafter called the "Tool") that converts Electronic Pocketbook application software written with LSI C (LSI C compiler Ver. 3.5 by LSI JAPAN Co., Ltd.) and its add-in software to C language programs with notation of a Hitachi-made compiler (Ver. 6.0).

This Tool only deals with the C language and does not support description of assembly language. Also, modification by the user may be needed depending on the program structure.

2. File structure of Tool

File structure of the Tool is shown in Table 1.

Table 1: File organization

File name	File description
Readme.txt	Notes on using Tool
Porting.exe	Execution file of Tool
Porting.pdf	Manual file in PDF format
help.html	HTML Help file for Tool
image (folder)	Folder of images for Help

3. Installation and Uninstallation

To install or uninstall the Tool, an installation application is not used.

Installation: Copy the files shown in Table 1 into the same folder.

Uninstallation: Delete the files shown in Table 1.

4. Overall Operation of Tool

The Tool performs conversion operation on C source files written with LSI C: (1) The Conversion Setting dialog allows you to select a language to be converted, batch or sequential conversion, and specify conversion items. (2) The File Selection dialog allows you to open the target file and execute the conversion. In case of sequential conversion, the Tool displays (3) the Confirm Conversion Item and (4) Confirm Conversion dialogs for each conversion and, when all the conversions are finished, displays (5) the Finish Conversion dialog.

In case of batch conversion, the Tool displays only (5) the Finish Conversion dialog after all the conversions have been finished. Figure 1 shows the flow of overall operation of the Tool.

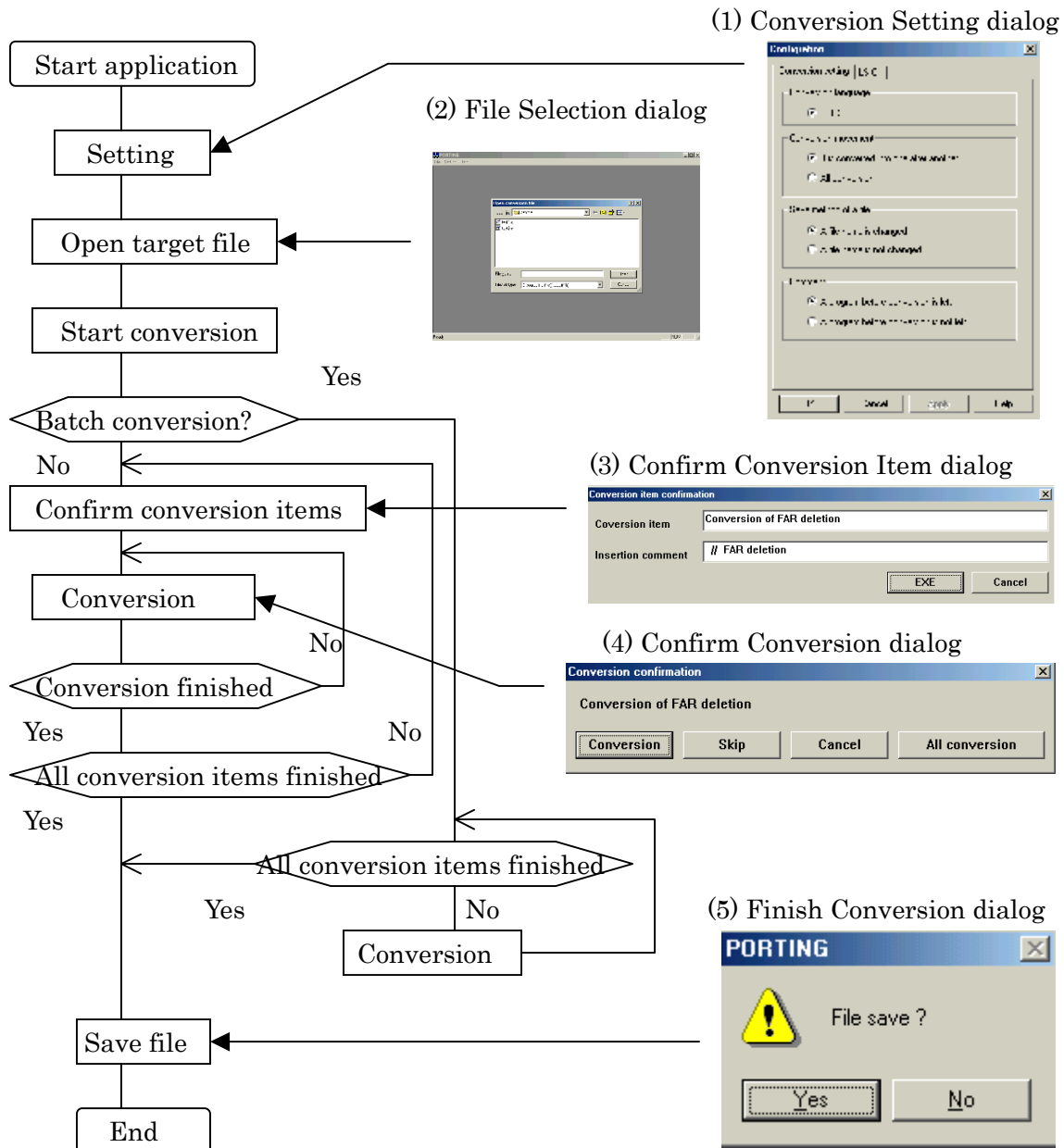


Figure 1: Flow of overall operation of Tool

5. Configuration of Tool's Main Frame Window

Figure 2 shows the main frame window of the Tool. The menu bar consists of the following items: File to read a file to be converted, Setting to make settings of the Tool, Execute to start conversion, Search and Replace to search and replace character strings in the window, Window to arrange windows and Help to display the version of the Tool.

Menu bar: File, Setting, Executing, Search, Window and Help

Windows for files to be converted

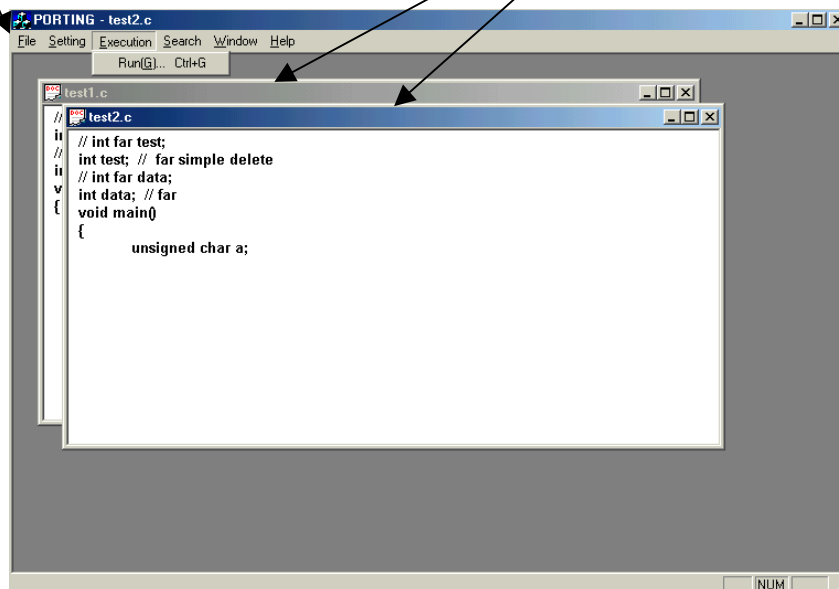


Figure 2: Structure of Tool's main frame window

6. Operation Procedure

6.1 Opening file

Run porting.exe and select File -> Open (Ctrl + O) to display the File Selection dialog box (fig. 3), and then select a file or files to be converted. (All of the opened files become targets for conversion. Multiple files can be selected at the same time.)

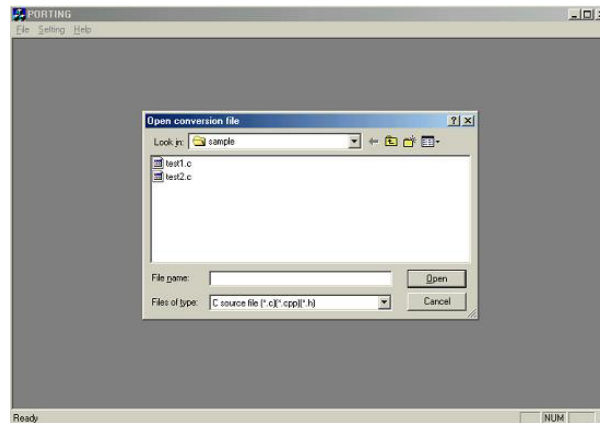


Figure 3: File Selection dialog box for conversion

6.2 How to make settings

Select Setting -> Detailed Setting (Ctrl + E) to display the Conversion Setting dialog. The Conversion Setting dialog consists of two tabs: Setting (fig. 4) and LSI C Setting (fig. 5).

(1) Setting tab

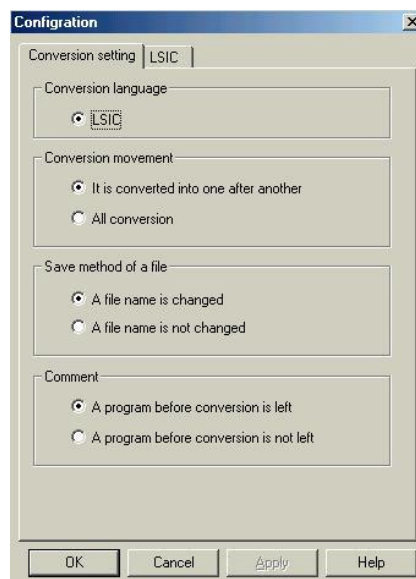


Figure 4: Setting tab

Conversion language:

Select the conversion language. The Tool performs conversion of the selected language.

- LSI C

Conversion movement:

Select the type of conversion movement: One after another or All conversion. If "One after another" is selected, the Confirm Conversion Item dialog (fig. 7) and the Confirm Conversion dialog (fig. 8) are displayed.

- It is converted into one after another
- All conversion

Save method of a file:

Select the method to save the converted file.

If "A file name is changed" is selected, the Tool adds "x" to the extension of the converted file and saves it in the same directory.

(Example) test.c -> test.cx

- A file name is changed
- A file name is not changed

Comment:

This is used to make a setting to comment out the conversion source. If "A program before conversion is left" is selected, the Tool leaves the source before conversion as a comment.

(Example) //int far test[10]; The Tool leaves the source before conversion as a comment.
int test[10];

- A program before conversion is left
- A program before conversion is not left

(2) LSI C Setting tab

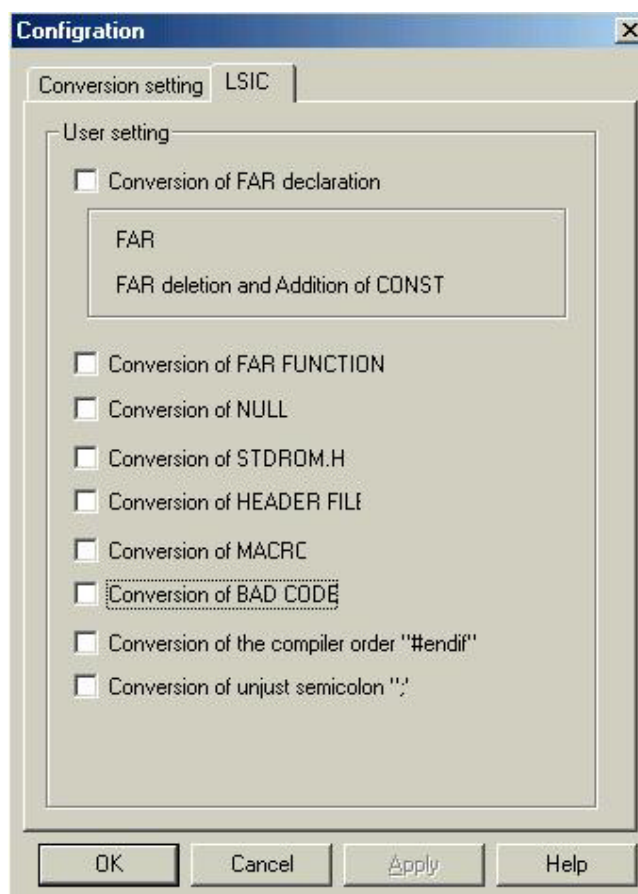


Figure 5: LSI C Setting tab

User settings:

This is used to select conversion items. Selected items become targets for conversion.

- Conversion of FAR declaration

FAR

FAR deletion and Addition of CONST

- Conversion of FAR FUNCTION
- Conversion of NULL
- Conversion of STDROM.H
- Conversion of HEADER FILE
- Conversion of MACRO
- Conversion of BAD CODE
- Conversion of the compiler order "#endif"
- Conversion of unjust semicolon ";"

6.3 Executing Tool

Select Execute to display the drop down menu (fig. 6).

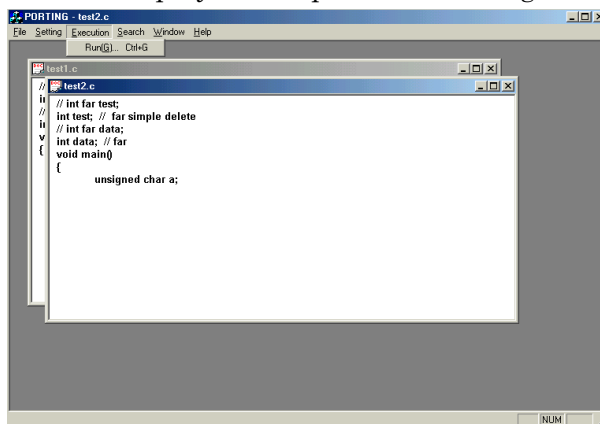


Figure 6: Drop down menu of Execute

When you select Run, a dialog is displayed to confirm the conversion item that you are performing and the comment to be inserted (fig. 7).

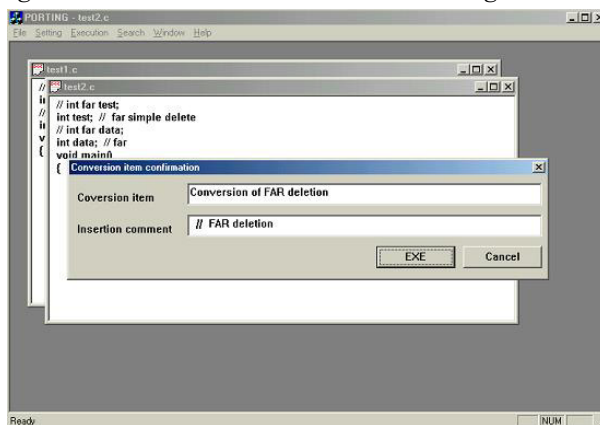


Figure 7: Confirm Conversion Item dialog

When you press the Execute button after the confirmation above, the Confirm Conversion dialog (fig. 8) is displayed.

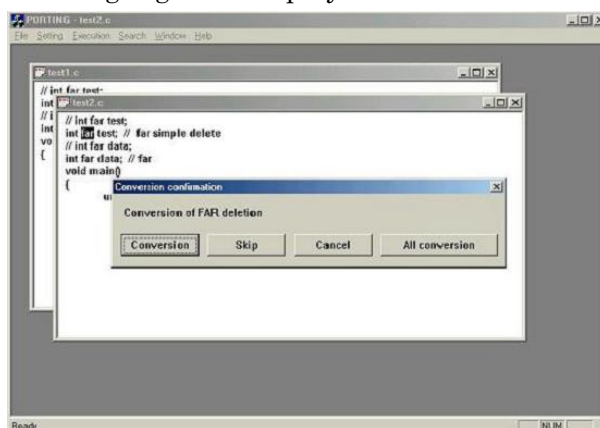


Figure 8: Confirm Conversion dialog

The Confirm Conversion dialog consists of Conversion, Skip, Cancel, and All Conversion.

Conversion: Performs conversion operation and searches for the next line to be converted.

Skip: Cancels conversion operation and searches for the next line to be converted.

Cancel: Stops conversion operation.

All Conversion: Performs batch conversion for all the files currently opened.

If there is a next conversion item when a conversion item is finished, the Confirm Conversion Item dialog (fig. 7) is displayed again. When all the conversion items are finished, the Finish Conversion dialog (fig. 9) is displayed.

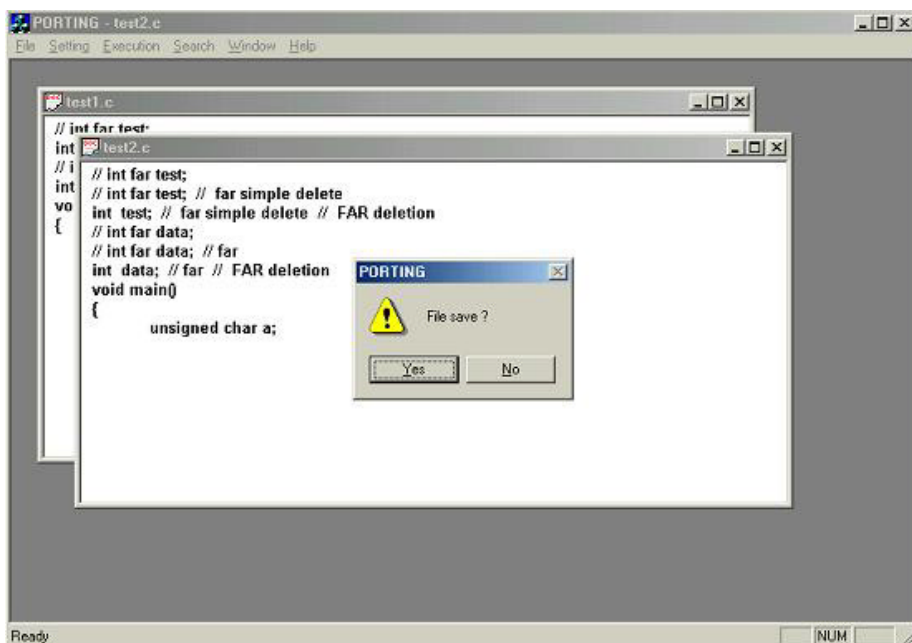


Figure 9: Finish Conversion dialog

The save file dialog consists of Yes and No.

Yes: Save all the files and close the dialog.

No: Not save files.

The Tool does not display here the "Save changes to xx?" dialog that appears when you exit the Tool by selecting No in the save file dialog that is displayed after execution of the Tool has finished. To save files after you have selected No in the save file dialog above, use the tool menu and select File, then Save as ... or Save.

7. Description of Tool's Functions (LSI C)

C language codes for LSI C conversion are shown in Table 2.

No	Conversion item	Description of conversion operation
1	Conversion of FAR declaration	Conversion of FAR declaration
2	Conversion of FAR FUNCTION	Conversion of name of FAR FUNCTION
3	Conversion of NULL	Conversion of NULL
4	Conversion of STDROM.H	Convert "stdrom.h" to "hitachi.h" Header file hitachi.h: - Add definition of size_t type - Add definition of PI(π): #define PI 3.1415926535897932 - Add functions for segmentation/composition of address: FP_SEG(), FP_OFF(), MK_FP() - Add functions to convert integer string: itoa(), ultoa(), ltoa() - Add software interrupt functions: int86(), int86x(), iny86y()
5	Conversion of HEADER FILE	Delete #include <malloc.h>, etc
6	Conversion of MACRO	Conversion of macro declaration
7	Conversion of BAD CODE	Convert illegal characters to codes
8	Conversion of compiler order "#endif"	Delete label of "#endif label"
9	Conversion of unjust semicolon ";"	Delete ";" in parts unnecessary for programming

Table 2: C language codes for LSI C conversion

8. Conversion Items of Porting Tool (LSI C)

8.1 Conversion of far declaration

[Differences between LSI C and Hitachi C compilers]

Far is a function that a standard C does not have, and unique to C for i8086 (LSI C).

The address space of i8086 is divided into two: "space that is smaller but efficiently accessible" and "space that is larger but costs you to access". And far is used as a keyword that is distributed in the larger address space. Hitachi C does not have this concept.

[Convert and review variables that used far declarations]

Search for a place where a far declaration was used and perform conversion. There are two types of conversion: one that simply deletes the far declaration; and one that deletes the far declaration and adds a const declaration.

[Sample conversion]

"Simply delete far declaration"

```
unsigned char far p;
```



```
unsigned char p;
```

```
work = ( char far * )Data[1];
```



```
work = ( char * )Data[1];
```

"Delete far declaration + add const declaration"

```
char far data[ ] = { " 1234 " };
```



```
char const data[ ] = { " 1234 " };
```

8.2 Conversion of FAR FUNCTION

[Differences between LSI C and Hitachi C compilers]

Because FAR does not exist in the Hitachi C compiler, a FAR function does not exist in it either.

[Description of conversion]

Convert a FAR function to a normal function.

[Sample conversion]

far_memcpy()



memcpy()

far_memmove()



memmove()

8.3 Conversion of NULL

[Differences between LSI C and Hitachi C compilers]

There is a difference in definition of NULL.

LSI C: #define NULL 0

Hitachi C: #define NULL ((void *) 0)

[Description of conversion]

Adopt LSI C's definition of NULL. Convert the part where NULL is used to value "0".

[Sample conversion]

int *p = NULL;



int *p = 0;

8.4 Conversion of STDROM.H

[Differences between LSI C and Hitachi C compilers]

A header file(STDROM.H) does not exist in Hitachi C.

[Description of conversion]

Create "hitachi.h", a header file in Hitachi C, that corresponds to "stdrom.h", LSI C's header file for that program. The Tool changes the header file name it reads.

[Sample conversion]

#include<stdrom.h> → #include<hitachi.h> (change file name)

8.5 Conversion of HEADER FILE

[Differences between LSI C and Hitachi C compilers]

There are differences between standard library header files that are used.

(Refer to Appendix: "Standard library header files".)

[Description of conversion]

Compare standard library header files used in LSI C and those used in Hitachi C, and convert the include statement of a header file that does not exist in Hitachi C into a comment.

[Sample conversion]

#include<malloc.h> → // #include<malloc.h> (convert into comment)
#include<farstr.h> → // #include<farstr.h> (convert into comment)

8.6 Conversion of MACRO

[Differences between LSI C and Hitachi C compilers]

There is a difference between the formats to undefine a macro declaration that has an argument.

LSI C: `#define identifier( identifier-list ) token-sequence`
 `#undef identifier( identifier-list )`

Hitachi C: `#define identifier( identifier-list ) token-sequence`
 `#undef identifier`

[Description of conversion]

Convert a line to undefine a macro declaration that has an argument into one in the format for the Hitachi C compiler.

[Sample conversion]

`#define MAX( A, B ) ((A) > (B) ? (A) : (B))`
`#undef MAX( A, B )`



`#define MAX( A, B ) ((A) > (B) ? (A) : (B))`
`#undef MAX`

8.7 Conversion of BAD CODE

[Differences between LSI C and Hitachi C compilers]

There are differences between characters that can be used.

(Refer to Appendix: "Character code table")

[Description of conversion]

Convert characters that cannot be used into codes.

[Sample conversion]

`char a[ ] = { "TEST ^O ^P" };`



`char a[ ] = { "TEST ¥0x00f ¥0x010" };`

8.8 Conversion of compiler order "#endif"

[Differences between LSI C and Hitachi C compilers]

There is a difference between the formats of compiler order "#endif".

LSI C: #if LABEL #endif LABEL

Hitachi C: #if LABEL #endif

[Description of conversion]

Convert a compiler control instruction "#endif" that uses LABEL into one in the format of the Hitachi C compiler.

[Sample conversion]

#if LABEL

#endif LABEL



#if LABEL

#endif // LABEL

8.9 Conversion of Unjust Semicolon ";"

[Differences between LSI C and Hitachi C compilers]

There is a difference between the ways to parse the dummy statement ";". The Hitachi C compiler outputs an error message regarding the dummy statement ";" that is outside a function declaration.

C2500 (E) Illegal token "phrase"

The sequence of the phrase does not conform to grammar.

[Description of conversion]

Delete the dummy statement ";" that is described outside a function declaration.

[Sample conversion]

void sbrk(int x)

{

};



void sbrk(int x)

{

}

9. Notes for Porting Tool (LSI C)

9.1 Output of warning due to incompatibility of type declarations

Because the level of error detection is different between the LSI C and Hitachi C compilers, a warning may be output in the latter due to an error that was not detected in the former. If there are no problems in operation under the following conditions, Hitachi C operates without any correction. The Porting Tool does not convert a warning that occurs regarding this description.

[Description of warning 1]

C1016 (W) Argument mismatch

The type of an argument in a prototype declaration and the type of corresponding argument in a function call are pointer types, and each points to a different type. Set the internal representation of the pointer in a function call as it is.

(Sample warning)

```
1:      void test(void);
2:      void sample( unsigned char * point, char data );    <- prototype declaration
3:      void test(void)
4:      {
5:          char work1[1];
6:          char work2 = 1 ;
7:          work1[0] = work2;
8:          sample( work1, work2 );
9:      }
10:     void sample( unsigned char * point, char data )
11:     {
12:         *point = data;
13:     }
```

Internal representation char

Internal representation unsigned char
--

* Line 8 is output of warning.

[Correction method]

Perform a cast conversion to match the types.

```
8:          sample( ( unsigned char * ) work1, work2 );
```

[Description of warning 2]

C1000 (W) Illegal pointer assignment

In an assignment between pointer types, the types that the pointer types point to are different from each other. Set the declaration of the assigned side as it is.

C1002 (W) Illegal pointer for "operator"

The operand of a binary operator, =, !=, >, <, >= or <=, does not point to a pointer type to the same type.

(Sample warning)

```
1:      void test(void);
3:      void test(void)
4:      {
5:          unsigned char * work1 ;
6:          char * work2 ;
7:          work2 = work1 ;
8:      }
```

* Line 7 is output of warning.

[Correction method]

Perform a cast conversion to match the types.

```
7:          work2 = ( char * )work1 ;
```

[Description of warning 3]

C1017 (W) Return type mismatch

The type returned by a function and the type of expression of a return statement are pointer types, and each points to a different type. Set the internal representation of the pointer in the expression of the return statement as it is.

(Sample of warning)

```
1:      void test(void);
2:      char * sample( void );
3:      void test(void)
4:      {
5:          char*point ;
6:          point = sample();
7:      }
8:      char * sample( void )
9:      {
10:         unsigned char*point ;
11:         return point ;
```

Internal representation char

Internal representation unsigned char
--

12: }

9.2 Treatment of floating-point constant

LSI C treats a floating-point constant as "long double type". Because the Hitachi C compiler treats it as "double type", the floating-point constant of the compiler is up to 17 digits. If the 17 digits are exceeded, the following warning is output. If the cause of warning is not corrected, the compiler performs calculations on values with reduced digits, therefore, the precision decreases. The Porting Tool does not convert a warning that occurs regarding this description.

[Description of warning]

C1024 (W) Out of float

The number of significant digits of floating-point constant exceeds 17.

[Correction method]

Reduce the number of digits of a floating-point constant that exceeds 17 to 17.

(Example) #define PI_2 3.14159265358979323846264338327950288



#define PI_2 3.1415926535897932

9.3 Use of assembly language

The Porting Tool does not perform conversion of assembly languages or assembly inline functions.

9.4 Initialization of self-reference structure

Because the Hitachi C compiler cannot specify a thing other than a constant expression for the initial value of a structure type, union type or array type variable, it outputs an error when another thing is specified. The Porting Tool does not convert an error that occurs regarding this description.

[Description of error]

2130 (E) Initializer should be constant

Specifying a thing other than a constant expression for the initial value of a structure type, union type or array type variable, or for the initial value of a statically allocated variable.

(Sample warning)

```
1: void main( void ){
2:     typedef struct a {
3:         int i;
4:         struct a *n;
5:     } AS;
6:     AS p[ ]={
7:         { 1, &p[1] },
8:         { 2, (void*) 0 }
9:     };
10: }
```

* Line 7 is output of warning.

[Correction method 1]

The error is avoided by defining local variables as static variables to let them have fixed addresses as described below. However, the variables are not generated onto the stack but reserve a memory area.

```
1: void main( void ){
2:     typedef struct a {
3:         int i;
4:         struct a *n;
5:     } AS;
6:     static AS p[ ]={          ← Correction
```

```

7:          { 1, &p[1] },
8:          { 2, (void*) 0 }
9:      };
10: }

```

[Correction method 2]

The error is avoided if reservation of the variable area and the initial value are not set at the same time, as described below. And the variables are generated onto the stack.

```

1: void main( void ){
2:     typedef struct a {
3:         int i;
4:         struct a *n;
5:     } AS;
6:     AS p[2];                                /* Reserve variable area */
7:     p[0].i = 1;                             /* Initialize      */
8:     p[0].n = &p[1] ;
9:     p[1].i = 2;
10:    p[1].n = (void*) 0;
11: }

```

9.5 Position to insert cast conversion

Hitachi C cannot perform a cast conversion at the head of a L-value. If it is used at that position, the following warning is output. The Porting Tool does not convert an error that occurs regarding this description.

[Description of warning]

C1022 (W) First operand of "operator" is not lvalue

The "operator" of the first operand cannot be a L-value.

(Sample warning)

```

1: unsigned int data;
2: (int) data -= -10;
3: if( data < 0 )
4: {
5: }

```

* Line 2 is output of warning.

[Correction method]

Add a cast conversion to the position where you want to use it signed or unsigned.

```
1: unsigned int data;  
2: data -= -10;  
3: if( (int) data < 0 )  
4: {  
5: }
```

9.6 Initialization of strings

Hitachi C cannot use a single and double quotations at the same time in specifying initialization of a variable. If they are used at the same time, the following error is output. The Porting Tool does not convert an error that occurs regarding this description.

[Description of error]

C2129 (E) Illegal initializer type

The initializer type is not one that can be assigned to a variable in specifying the initializer of a variable.

(Sample error)

```
1: char mess[ ] = { ' ',0xf6,0xa1,'/','4',0xf6,0xb0,"R" };
```

[Correction method]

```
1: char mess[ ] = { " /x06/xa1¥/4/xf6/xb0R" };
```

9.7 FP_OFF(), FP_SEG() and MK_FP() - segmentation/composition of far pointer [Differences between LSI C and Hitachi C compilers]

A far pointer used in LSI C consists of 16-bit segments plus 16-bit offsets, 32 bits in total (segments are high order, offsets low order). LSI C uses FP_OFF(), FP_SEG() and MK_FP() for segmentation and composition of this far pointer.

As for pointers of Hitachi C, there is not a concept of segment and offset and FP_OFF(), FP_SEG() and MK_FP() do not exist.

FP_OFF(): Get offset

Gets the offset value of a far pointer and returns it as an unsigned type.

FP_SEG(): Get segment

Gets the segment value of a far pointer and returns it as an unsigned type.

MK_FP(): Make address

Makes a far pointer from the offset and segment values and returns it.

[Description of conversion]

The concept for pointers in LSI C is used in Hitachi C as well. Thus Hitachi C is supported without performing conversion for an LSI C program that uses segmentation and composition of far pointers. To realize this, we added FP_OFF(), FP_SEG() and MK_FP() that operate in the same way as those for LSI C as macro definitions.

[Added macro definitions]

```
#define FP_SEG(p) ((unsigned short)((unsigned long)((void*)(p)) >> 16))
#define FP_OFF(p) (unsigned short)(p)
#define MK_FP(s, o) ((void*)((((unsigned long)(s) << 16) | (unsigned short)(o))))
```

[Supplement]

(Differences between addresses of 8086 CPU and Hitachi CPU)

An 8086 CPU uses an addressing technique called the segment method, using dedicated registers called segment registers. A Hitachi CPU has no segment registers and has direct access to an address space up to 4 Gbytes (32-bit "SH").

Segment method:

This was contrived to keep the compatibility with 16-bit addressing (64-Kbyte address space) and extend the address space (1 Mbyte) at the same time. But the address space that is linearly accessible is limited to 64 Kbytes, and dedicated registers called segment registers are used to access a space exceeding the size.

Table 3: Character code table

Char	Code	Desc	Hitachi	LSIC	Char	Code	Desc	Hitachi	LSIC
^@	0x00	NULL	X	X		0x20	SP	O	O
^A	0x01	SOH	X	O	!	0x21	-	O	O
^B	0x02	STX	X	O	«	0x22	-	O	O
^C	0x03	ETX	X	O	#	0x23	-	O	O
^D	0x04	EOT	X	O	\$	0x24	-	O	O
^E	0x05	ENQ	X	O	%	0x25	-	O	O
^F	0x06	ACK	X	O	&	0x26	-	O	O
^G	0x07	BEL	X	O	‘	0x27	-	O	O
^H	0x08	BS	X	O	(	0x28	-	O	O
^I	0x09	HT	X	O	)	0x29	-	O	O
^J	0x0A	LF	X	O	*	0x2A	-	O	O
^K	0x0B	VT	X	O	+	0x2B	-	O	O
^L	0x0C	FF	X	O	,	0x2C	-	O	O
^M	0x0D	CR	X	O	-	0x2D	-	O	O
^N	0x0E	SO	X	O	.	0x2E	-	O	O
^O	0x0F	SI	X	O	/	0x2F	-	O	O
^P	0x10	DLE	X	O	0	0x30	-	O	O
^Q	0x11	DC1	X	O	1	0x31	-	O	O
^R	0x12	DC2	X	O	2	0x32	-	O	O
^S	0x13	DC3	X	O	3	0x33	-	O	O
^T	0x14	DC4	X	O	4	0x34	-	O	O
^U	0x15	NAK	X	O	5	0x35	-	O	O
^V	0x16	SYN	X	O	6	0x36	-	O	O
^W	0x17	ETB	X	O	7	0x37	-	O	O
^X	0x18	CAN	X	O	8	0x38	-	O	O
^Y	0x19	EM	X	O	9	0x39	-	O	O
^Z	0x1A	SUB	X	O	:	0x3A	-	O	O
^[	0x1B	ESC	X	O	;	0x3B	-	O	O
^¥	0x1C	FS	X	O	<	0x3C	-	O	O
^]	0x1D	GS	X	O	=	0x3D	-	O	O
^^	0x1E	RS	X	O	>	0x3E	-	O	O
^_	0x1F	US	X	O	?	0x3F	-	O	O

Char	Code	Desc	Hitachi	LSIC	Char	Code	Desc	Hitachi	LSIC
@	0x40	-	O	O	`	0x60	-	O	O
A	0x41	-	O	O	a	0x61	-	O	O
B	0x42	-	O	O	b	0x62	-	O	O
C	0x43	-	O	O	c	0x63	-	O	O
D	0x44	-	O	O	d	0x64	-	O	O
E	0x45	-	O	O	e	0x65	-	O	O
F	0x46	-	O	O	f	0x66	-	O	O
G	0x47	-	O	O	g	0x67	-	O	O
H	0x48	-	O	O	h	0x68	-	O	O
I	0x49	-	O	O	i	0x69	-	O	O
J	0x4A	-	O	O	j	0x6A	-	O	O
K	0x4B	-	O	O	k	0x6B	-	O	O
L	0x4C	-	O	O	l	0x6C	-	O	O
M	0x4D	-	O	O	m	0x6D	-	O	O
N	0x4E	-	O	O	n	0x6E	-	O	O
O	0x4F	-	O	O	o	0x6F	-	O	O
P	0x50	-	O	O	p	0x70	-	O	O
Q	0x51	-	O	O	q	0x71	-	O	O
R	0x52	-	O	O	r	0x72	-	O	O
S	0x53	-	O	O	s	0x73	-	O	O
T	0x54	-	O	O	t	0x74	-	O	O
U	0x55	-	O	O	u	0x75	-	O	O
V	0x56	-	O	O	v	0x76	-	O	O
W	0x57	-	O	O	w	0x77	-	O	O
X	0x58	-	O	O	x	0x78	-	O	O
Y	0x59	-	O	O	y	0x79	-	O	O
Z	0x5A	-	O	O	z	0x7A	-	O	O
[	0x5B	-	O	O	{	0x7B	-	O	O
¥	0x5C	-	O	O		0x7C	-	O	O
]	0x5D	-	O	O	}	0x7D	-	O	O
_	0x5E	-	O	O		0x7E	-	O	O
	0x5F	-	O	O		0x7F	DEL	X	O

Table 4: Standard library header files

No	Standard library header file	Hitach	LSIC	Description of conversion
1	Stdrom.h	X	O	Change file name to hitachi.h
2	Ctype.h	O	O	
3	Errno.h	O	O	
4	Farstr.h	X	O	comment (substitute by string.h)
5	Limits.h	O	O	
6	Float.h	O	O	
7	Machine.h	O	O	
8	Malloc.h	X	O	comment (substitute by stdlib.h)
9	Math.h	O	O	
10	Setjmp.h	O	O	
11	Stdarg.h	O	O	
12	Stddef.h	O	O	
13	String.h	O	O	
14	Stdlib.h	O	O	

(o: File, x: No file)